

Project	– Aavin IWS
Device	- AWS MINI
Concept	- IN/OUT
Camera Used	- Prama (Hikvision)
Camera Count	- 4
Software Version	– 3.16A
Interface Sensor.	- RFID2, Display, Audio System, Photo Electric
RFID Reader Count	– 2 (IN & OUT)
Indicator (Signal) Count	– 2 (IN & OUT)
Site Name	– Aavin (Site 1 - Demo)
Site Location	– Sathiyamangalam
Demo Date	– 24 / NOV / 2025

Uno Logic Code:

```
/* *****  
 * RS485 DISPLAY INTERFACE - FINAL VERSION  
 *  
 * Dual Display Support:  
 * Display #1 ID = 0x01  
 * Display #2 ID = 0x02  
 *  
 * RED Command = 2-Stage Sequence:  
 *  
 * Stage 1 (0 sec):  
 * <L3><F1><S1><R><Stop in WB, "VEHICLE" >  
 *  
 * Stage 2 (after 20 sec):  
 * <L3><F1><S1><K><"VEHICLE" & "WEIGHT">  
 *  
 * Both repeat every 1 second.  
 ***** */  
  
#include <SoftwareSerial.h>  
  
// ---- Signal Lights ----  
#define RED_PIN    A1  
#define GREEN_PIN  A0  
#define ORG_PIN    A2  
  
// ---- RS485 Pins ----  
#define RS485_RE    2  
#define RS485_DE    3  
#define RS485_R0    10  
#define RS485_DI    11  
  
// ---- Software Serial for RS485 ----  
SoftwareSerial rs485(RS485_R0, RS485_DI); // RX, TX  
  
// ---- RAW BYTE HEADER (ID byte replaced at runtime) ----  
byte header[] = {0x01, 0x10, 0x00, 0x00, 0x00, 0x20, 0x40};  
  
// ---- Message System ----  
String lastMessage = "";  
String lastVehicle = "";  
String lastWeight  = "";  
  
unsigned long lastSendTime = 0;  
unsigned long redStageStartTime = 0;  
  
const unsigned long SEND_INTERVAL = 1000; // repeat every 1 second
```

```

const unsigned long RED_STAGE_TIME = 5000; // 20 sec delay

// =====
// Send RAW bytes to a specific display ID
// =====
void sendToDisplayID(byte displayID, const String &asciiText) {

    digitalWrite(RS485_RE, HIGH);
    digitalWrite(RS485_DE, HIGH);

    // Send header (first byte replaced with display ID)
    for (int i = 0; i < sizeof(header); i++) {
        if (i == 0)
            rs485.write(displayID);
        else
            rs485.write(header[i]);
    }

    // Send ASCII text
    for (int i = 0; i < asciiText.length(); i++) {
        rs485.write((byte)asciiText[i]);
    }

    digitalWrite(RS485_RE, LOW);
    digitalWrite(RS485_DE, LOW);
}

// =====
// Send SAME message to BOTH displays
// =====
void sendToDisplay(const String &asciiText) {
    sendToDisplayID(0x01, asciiText); // Display #1
    sendToDisplayID(0x02, asciiText); // Display #2
}

void setup() {

    pinMode(RED_PIN, OUTPUT);
    pinMode(GREEN_PIN, OUTPUT);
    pinMode(ORG_PIN, OUTPUT);

    pinMode(RS485_RE, OUTPUT);
    pinMode(RS485_DE, OUTPUT);

    digitalWrite(RED_PIN, LOW);
    digitalWrite(GREEN_PIN, LOW);
}

```

```

    digitalWrite(ORG_PIN, LOW);

    digitalWrite(RS485_RE, LOW);
    digitalWrite(RS485_DE, LOW);

    Serial.begin(9600);
    rs485.begin(9600);
}

// =====
// MAIN LOOP
// =====
void loop() {

    // -----
    // 1) Check for NEW command from CPU (via Serial)
    // -----
    if (Serial.available()) {

        String cmd = Serial.readStringUntil('\n');
        cmd.trim();

        // -----
        // GREEN
        // -----
        if (cmd == "GRN") {

            digitalWrite(GREEN_PIN, HIGH);
            digitalWrite(RED_PIN, LOW);
            digitalWrite(ORG_PIN, LOW);

            // RESET RED MESSAGE VARIABLES
            lastVehicle = "";
            lastWeight = "";
            redStageStartTime = 0;

            lastMessage = "<L3><F1><S1><W><Ready...!!! >";

            sendToDisplay(lastMessage);
            lastSendTime = millis();
        }

        // -----
        // ORANGE
        // -----
        else if (cmd == "ORG") {

            digitalWrite(ORG_PIN, HIGH);

```

```

digitalWrite(GREEN_PIN, LOW);
digitalWrite(RED_PIN, LOW);

// RESET RED MESSAGE VARIABLES
lastVehicle = "";
lastWeight = "";
redStageStartTime = 0;

lastMessage = "<L3><F1><S1><G><Process Completed, GO...!!! >";

sendToDisplay(lastMessage);
lastSendTime = millis();
}

// -----
// RED (2-stage message)
// -----
else if (cmd.startsWith("RED")) {

    digitalWrite(RED_PIN, HIGH);
    digitalWrite(GREEN_PIN, LOW);
    digitalWrite(ORG_PIN, LOW);

    // Extract vehicle + weight
    String data = cmd.substring(4);
    int p = data.lastIndexOf(' ');

    lastWeight = data.substring(p + 1);
    lastVehicle = data.substring(0, p);

    // ---- Stage 1 Message ----
    lastMessage = "<L3><F1><S1><R><Stop in WB, \";
    lastMessage += lastVehicle;
    lastMessage += "\" >";

    sendToDisplay(lastMessage);

    lastSendTime = millis();
    redStageStartTime = millis();
}

}

// -----
// 2) After 20 seconds → switch RED Stage-1 to Stage-2
// -----
if (lastVehicle.length() > 0) {

    if (millis() - redStageStartTime >= RED_STAGE_TIME &&

```

```

redStageStartTime != 4294967295) {

    // ---- Stage 2 Message ----
    lastMessage = "<L3><F1><S1><K><\"";
    lastMessage += lastVehicle;
    lastMessage += "\" & \"";
    lastMessage += lastWeight;
    lastMessage += "\">";

    // disable further switching
    redStageStartTime = 4294967295;
}

}

// -----
// 3) Repeat the current message every 1 second
// -----
if (lastMessage.length() > 0) {

    if (millis() - lastSendTime >= SEND_INTERVAL) {
        sendToDisplay(lastMessage);
        lastSendTime = millis();
    }
}

}

```









